FIAP

FERNANDO OLIVEIRA FRANCELINO SARDINHA - RM570196

JONATAN VIEIRA FEITOSA – RM570452

MARCELO CANDIDO DA MATA JUNIOR – RM569584

VINICIUS LIBERATO DOS ANJOS – RM571480

DAVI FELIX CUNHA - RM569393

CHALLENGE TDSPY: TETRAMERGE CONSULTING (TM CONSULTING)

São Paulo

2026

RESUMO

O projeto SoulUp consiste no desenvolvimento de uma solução gamificada voltada ao incentivo e reconhecimento de práticas sustentáveis, utilizando conceitos de Domain Driven Design e Programação Orientada a Objetos em Java. Nesta etapa, a aplicação foi estruturada em camadas responsáveis pela representação das entidades do domínio, aplicação das regras de negócio, persistência dos dados e comunicação com o banco de dados Oracle.

A implementação contempla classes de modelo relacionadas aos usuários, ações sustentáveis e registros de ações, além de métodos responsáveis por validações, cálculo e atribuição de pontuação e geração de ranking. Para a persistência, foram desenvolvidas classes DAO com operações completas de Create, Read, Update e Delete (CRUD), integradas ao banco de dados por meio de uma classe específica de conexão.

O projeto também dispõe de uma classe de teste com método main, utilizada para instanciar objetos e simular as principais funcionalidades da solução, incluindo operações de persistência e execução das regras de negócio. A modelagem é complementada pelo Modelo Entidade-Relacionamento (MER) e pelo Diagrama de Classes UML, proporcionando uma visão integrada entre a estrutura do banco de dados e a implementação orientada a objetos.

Dessa forma, esta Sprint consolida a evolução técnica do SoulUp ao integrar modelagem de domínio, orientação a objetos, regras de negócio e persistência de dados, estabelecendo uma base estruturada para a continuidade e expansão das funcionalidades da solução nas próximas etapas do projeto.

Sumário

1. OBJETIVO E ESCOPO 4

1.1. ESCOPO DESTA ENTREGA 4

2. FUNCIONALIDADES E REGRAS DE NEGÓCIO 5

PROTÓTIPO DO SISTEMA 6

3. ESTRUTURA JAVA E DIAGRAMA DE CLASSES 6

3.1. Relações Principais 7

3.2. Diagrama de Classes 7

3.3. Critérios de representação do diagrama 8

4. MODELAGEM DO BANCO DE DADOS 9

4.1. Script SQL DDL Completo 10

5. INTEGRAÇÃO JAVA X BANCO DE DADOS 15

6. CONEXÕES E FECHAMENTO DE RECURSOS 16

6.1. Fechamento explicito 16

7. DAO E CRUD COMPLETO 17

8. PROCEDIMENTOS PARA EXECUTAR 17

8.1. Execução do projeto 17

8.2. Oracle oficial 17

9. VALIDAÇÃO FINAL E OBSERVAÇÕES 18

10. CONCLUSÃO 18

11. REFERÊNCIAS 18

1. OBJETIVO E ESCOPO

A solução consiste em uma plataforma integrada à SoulUp-Prospera na qual o usuário seleciona uma ação sustentável, realiza a atividade e envia evidências, como fotos, vídeos, geolocalização ou comprovantes. O sistema processa essas informações, aplica regras de negócio e mecanismos de validação e, quando a ação é aprovada, atribui pontos ao usuário.

O Sistema de Gamificação Sustentável foi desenvolvido para incentivar ações sustentáveis por meio de pontuação e ranking. Nesta Sprint 3, o projeto Java foi evoluído para incluir persistência de dados, conexão JDBC, camada DAO, CRUD completo, encapsulamento e uma classe Main exclusiva para testes.

O banco de dados utilizado como referência é o modelo físico oficial entregue pela equipe na disciplina Building Relational Database. O Java utiliza diretamente as entidades centrais TB_USUARIO, TB_ACAO_SUSTENTAVEL e TB_REGISTRO_ACAO, mantendo os mesmos nomes de tabelas e colunas definidos no script Oracle.

1.1.ESCOPO DESTA ENTREGA

- Cadastro, consulta, alteração e exclusão de usuários.
- Cadastro, consulta, alteração e exclusão de ações sustentáveis.

- Registro, consulta, alteração e exclusão das ações realizadas.
- Validação de dados antes das operações.
- Cálculo de pontuação conforme dificuldade da ação.
- Atualização da pontuação do usuário e exibição de ranking.
- Integração com o banco Oracle oficial da FIAP por meio de JDBC.

2. FUNCIONALIDADES E REGRAS DE NEGÓCIO

As regras foram mantidas simples e compatíveis com o nível trabalhado em aula.

Regra	Implementação
Validação do usuário	Nome obrigatório, e-mail com @, CPF com 11 caracteres, senha, cidade e bairro preenchidos e pontos não negativos.
Validação da ação	Nome e objetivo obrigatórios, pontuação base maior que zero e dificuldade BAIXA, MEDIA ou ALTA.
Cálculo de pontuação	BAIXA = pontuação base; MEDIA = base + 10; ALTA = base + 20.
Registro de ação	Gera um registro PENDENTE, associa usuário e ação e guarda os pontos obtidos.
Ranking	Ordena usuários da maior para a menor pontuação usando estrutura simples de repetição.

Métodos relevantes incluem validarCadastro(), adicionarPontos(), validarAcao(), calcularPontuacao(), registrarAcao() e ordenar(). Dessa forma, o projeto supera o mínimo de quatro métodos com lógica de negócio solicitado na Sprint.

PROTÓTIPO DO SISTEMA

Os protótipos abaixo representam telas planejadas para a solução e demonstram como o usuário interagiria com as funcionalidades do Java.

Tela 1 - Ranking

1º Ana Souza - 140 pontos
2º Pedro Silva - 50 pontos
[Registrar nova ação]
O usuário acompanha sua posição e acessa o registro de uma nova atividade sustentável.

Tela 2 Registrar Ação

Ação: Reciclagem de materiais
Objetivo sustentável: Consumo responsável
Dificuldade: [BAIXA | MEDIA | ALTA]
[Validar e registrar]
O sistema valida a ação, calcula a pontuação e cria o registro.

Tela 3 Histórico

Data/Hora | Ação | Status | Pontos
12/09/2026 | Reciclagem | VALIDADO | 50
O usuário consulta os registros e os pontos obtidos.

3. ESTRUTURA JAVA E DIAGRAMA DE CLASSES

A estrutura do projeto separa conexão, persistência, interfaces, modelos, serviços e testes.

src/main/java/

conexao/ ConexaoOracleProjeto.java

dao/ UsuarioDAO.java, AcaoSustentavelDAO.java, RegistroAcaoDAO.java

interfaces/ Calculavel.java

model/ Usuario.java, AcaoSustentavel.java, RegistroAcao.java

service/ Ranking.java, SistemaGamificacao.java

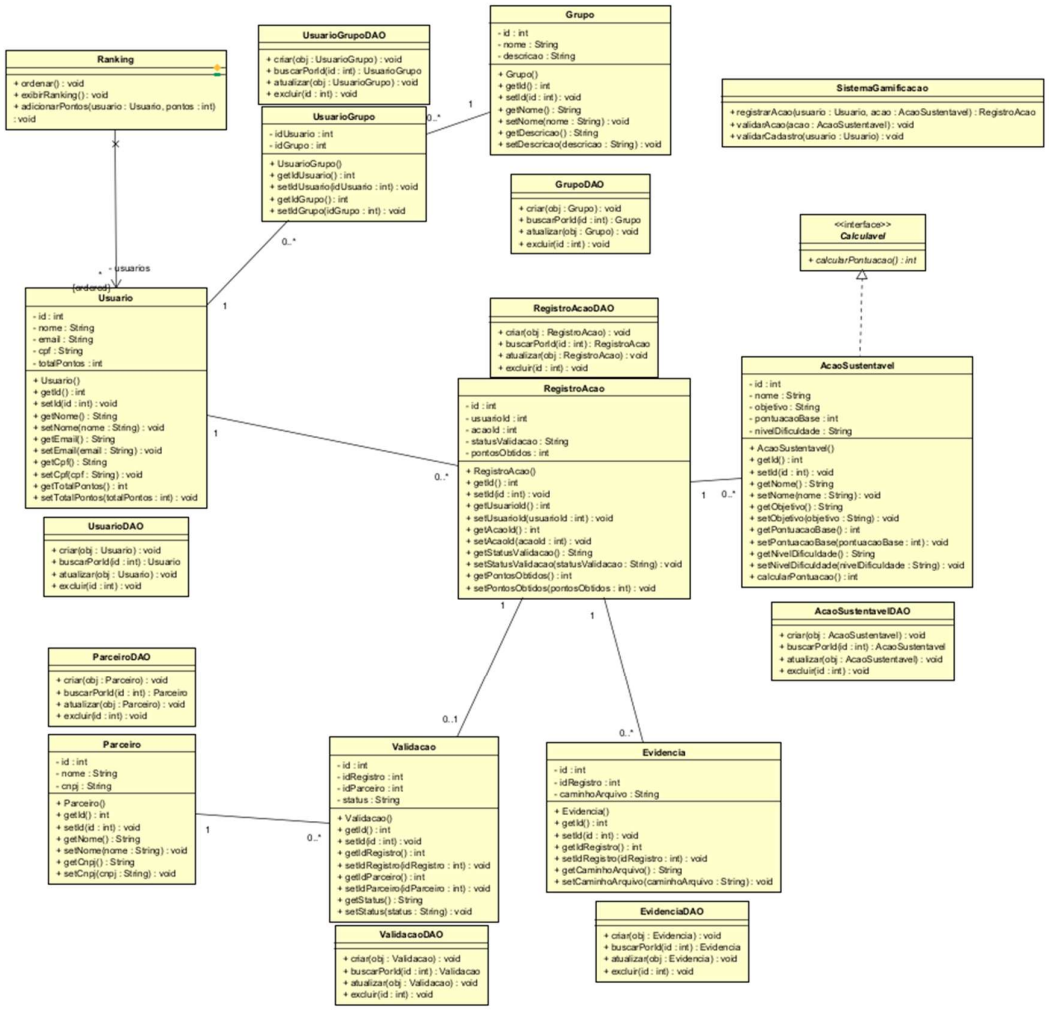
teste/ Main.java

3.1.Relações Principais

AcaoSustentavel implementa a interface Calculavel. SistemaGamificacao utiliza Usuario e AcaoSustentavel para gerar RegistroAcao. Ranking mantém uma lista de Usuario. Cada DAO é responsável pela persistência de uma classe modelo.

3.2.Diagrama de Classes

O diagrama de classes UML foi elaborado no Astah a partir da estrutura real do projeto Java e mantido no arquivo soulUp.asta. A representação contempla os pacotes, classes, atributos e métodos implementados, com destaque para as camadas model, service, dao, conexao, interfaces e teste:



3.3.Critérios de representação do diagrama

A inclusão das classes DAO no diagrama atende diretamente à exigência de que o Diagrama de Classes seja atualizado conforme a estrutura da aplicação. Como UsuarioDAO, AcaoSustentavelDAO e RegistroAcaoDAO são classes Java reais e concentram o CRUD completo avaliado na Sprint, sua presença no diagrama demonstra de forma objetiva a correspondência entre a documentação e a implementação entregue.

Arquiteturalmente, a organização por pacotes evidencia a separação de responsabilidades entre as regras de negócio, representadas principalmente por model e service, e o acesso ao banco de dados, concentrado em dao e conexao. Os métodos criar, buscarPorId, listar, atualizar e excluir apresentados nos DAOs tornam visível ao avaliador a implementação da camada de persistência e do CRUD.

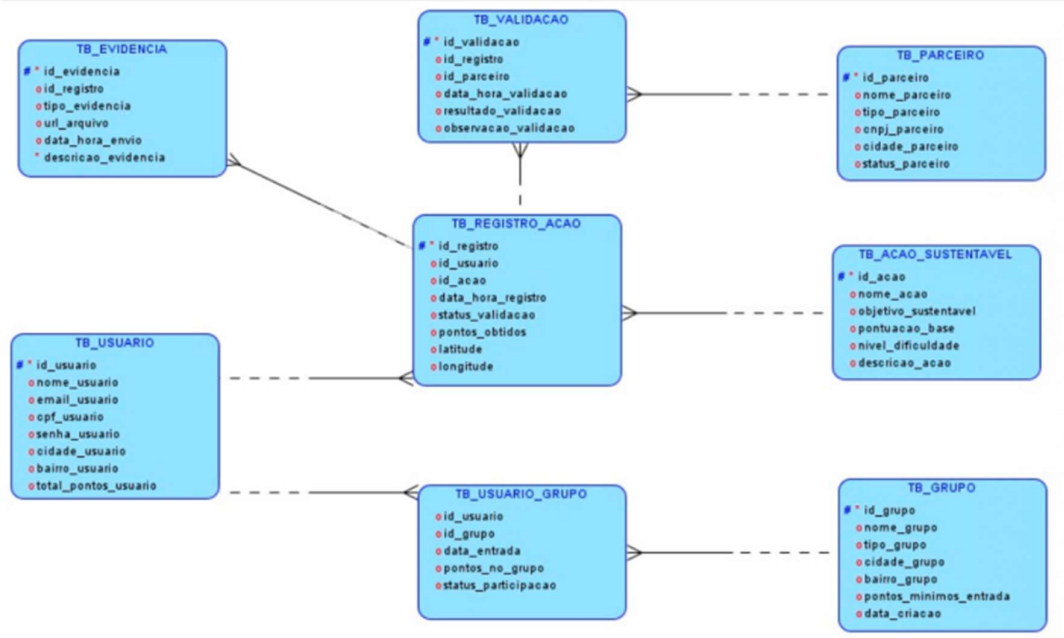
Cada DAO gerencia exclusivamente sua entidade correspondente: UsuarioDAO trabalha com Usuario, AcaoSustentavelDAO com AcaoSustentavel e RegistroAcaoDAO com RegistroAcao. Em uma representação UML exaustiva, essas utilizações também poderiam ser mostradas por dependências tracejadas, pois as entidades aparecem como parâmetros e retornos dos métodos. Para preservar a legibilidade do diagrama e evitar excesso de linhas cruzadas, foram priorizadas as relações essenciais e a proximidade visual entre elementos relacionados, sem alterar a estrutura efetivamente implementada no código.

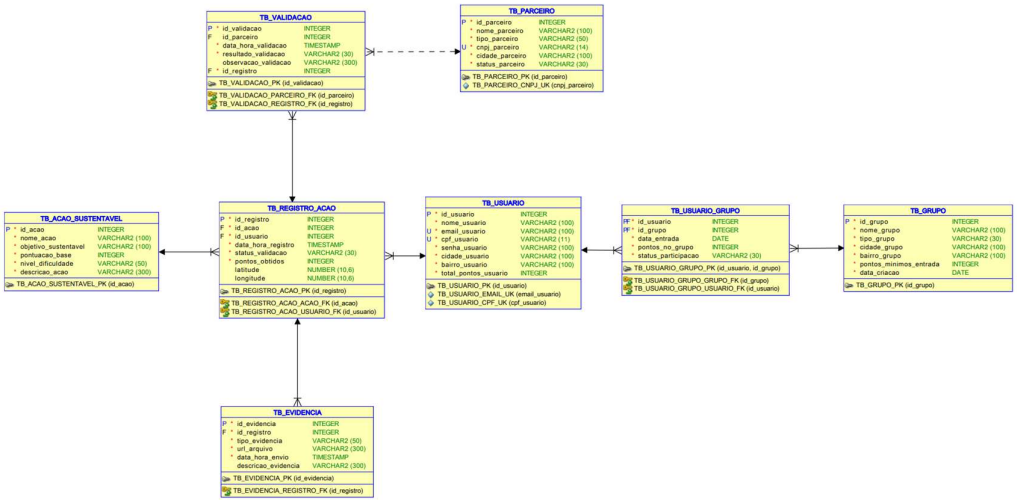
4. MODELAGEM DO BANCO DE DADOS

O script físico recebido da equipe possui oito tabelas Oracle e utiliza PK, FK, UNIQUE, CHECK e NOT NULL.

Tabela	Finalidade
TB_USUARIO	Dados cadastrais e total de pontos.
TB_GRUPO	Grupos de participação dos usuários.
TB_PARCEIRO	Parceiros responsáveis por validações.
TB_ACAO_SUSTENTAVEL	Catálogo de ações e pontuação base.
TB_USUARIO_GRUPO	Relacionamento N:M entre usuário e grupo.
TB_REGISTRO_ACAO	Ação realizada pelo usuário, status e pontos.
TB_EVIDENCIA	Evidências anexadas ao registro.
TB_VALIDACAO	Resultado da validação feita por parceiro.

Relacionamentos centrais: TB_USUARIO 1:N TB_REGISTRO_ACAO; TB_ACAO_SUSTENTAVEL 1:N TB_REGISTRO_ACAO; TB_REGISTRO_ACAO 1:N TB_EVIDENCIA; TB_REGISTRO_ACAO 1:1 TB_VALIDACAO; TB_USUARIO N:M TB_GRUPO por TB_USUARIO_GRUPO; TB_PARCEIRO 1:N TB_VALIDACAO





4.1.Script SQL DDL Completo

Abaixo está o script completo estruturado para o Oracle Database. Ele contém os blocos de deleção para limpeza do schema, criação das tabelas obedecendo à hierarquia de dependências e as definições de constraints obrigatórias (PK, FK, UNIQUE, CHECK e NOT NULL).

```
DROP TABLE TB_VALIDACAO CASCADE CONSTRAINTS;
DROP TABLE TB_EVIDENCIA CASCADE CONSTRAINTS;
DROP TABLE TB_REGISTRO_ACAO CASCADE CONSTRAINTS;
DROP TABLE TB_USUARIO_GRUPO CASCADE CONSTRAINTS;
DROP TABLE TB_ACAO_SUSTENTAVEL CASCADE CONSTRAINTS;
DROP TABLE TB_PARCEIRO CASCADE CONSTRAINTS;
DROP TABLE TB_GRUPO CASCADE CONSTRAINTS;
DROP TABLE TB_USUARIO CASCADE CONSTRAINTS;

CREATE TABLE TB_USUARIO (
id_usuario INTEGER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1),
nome_usuario VARCHAR2(100) NOT NULL,
email_usuario VARCHAR2(100) NOT NULL,
cpf_usuario VARCHAR2(11) NOT NULL,
senha_usuario VARCHAR2(100) NOT NULL,
cidade_usuario VARCHAR2(100) NOT NULL,
bairro_usuario VARCHAR2(100) NOT NULL,
total_pontos_usuario INTEGER DEFAULT 0 NOT NULL,
CONSTRAINT PK_TB_USUARIO PRIMARY KEY (id_usuario),
CONSTRAINT UK_USUARIO_CPF UNIQUE (cpf_usuario),
CONSTRAINT UK_USUARIO_EMAIL UNIQUE (email_usuario),
CONSTRAINT CK_USUARIO_PONTOS CHECK (total_pontos_usuario >= 0)
);

CREATE TABLE TB_GRUPO (
```

```
id_grupo INTEGER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1),
nome_grupo VARCHAR2(100) NOT NULL,
tipo_grupo VARCHAR2(30) NOT NULL,
cidade_grupo VARCHAR2(100),
bairro_grupo VARCHAR2(100),
pontos_minimos_entrada INTEGER DEFAULT 0 NOT NULL,
data_criacao DATE DEFAULT SYSDATE NOT NULL,
CONSTRAINT PK_TB_GRUPO PRIMARY KEY (id_grupo),
CONSTRAINT CK_GRUPO_TIPO CHECK (tipo_grupo IN ('ABERTO', 'PRIVADO')),
CONSTRAINT CK_GRUPO_PTS_MIN CHECK (pontos_minimos_entrada >= 0)
);

CREATE TABLE TB_PARCEIRO (
id_parceiro INTEGER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1),
nome_parceiro VARCHAR2(100) NOT NULL,
tipo_parceiro VARCHAR2(50) NOT NULL,
cnpj_parceiro VARCHAR2(14) NOT NULL,
cidade_parceiro VARCHAR2(100) NOT NULL,
status_parceiro VARCHAR2(30) DEFAULT 'ATIVO' NOT NULL,
CONSTRAINT PK_TB_PARCEIRO PRIMARY KEY (id_parceiro),
CONSTRAINT UK_PARCEIRO_CNPJ UNIQUE (cnpj_parceiro),
 CONSTRAINT CK_PARCEIRO_STATUS CHECK (status_parceiro IN ('ATIVO', 'INATIVO', 'EM_ANALISE'))
 );
```

```
CREATE TABLE TB_ACAO_SUSTENTAVEL (

id_acao INTEGER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1),

nome_acao VARCHAR2(100) NOT NULL,

objetivo_sustentavel VARCHAR2(100) NOT NULL,

pontuacao_base INTEGER NOT NULL,

nivel_dificuldade VARCHAR2(50) NOT NULL,

descricao_acao VARCHAR2(300),

CONSTRAINT PK_TB_ACAO_SUSTENTAVEL PRIMARY KEY (id_acao),

CONSTRAINT CK_ACAO_PONTUACAO CHECK (pontuacao_base > 0),

CONSTRAINT CK_ACAO_DIFICULDADE CHECK (nivel_dificuldade IN ('BAIXA', 'MEDIA', 'ALTA'))

 );


CREATE TABLE TB_USUARIO_GRUPO (

id_usuario INTEGER NOT NULL,

id_grupo INTEGER NOT NULL,

data_entrada DATE DEFAULT SYSDATE NOT NULL,

pontos_no_grupo INTEGER DEFAULT 0 NOT NULL,

status_participacao VARCHAR2(30) DEFAULT 'ATIVO' NOT NULL,

CONSTRAINT PK_TB_USUARIO_GRUPO PRIMARY KEY (id_usuario, id_grupo),

CONSTRAINT FK_US_GRP_USUARIO FOREIGN KEY (id_usuario) REFERENCES TB_USUARIO (id_usuario),

CONSTRAINT FK_US_GRP_GRUPO FOREIGN KEY (id_grupo) REFERENCES TB_GRUPO (id_grupo),

CONSTRAINT CK_US_GRP_STATUS CHECK (status_participacao IN ('ATIVO', 'INATIVO', 'SUSPENSO'))
```

```
);

CREATE TABLE TB_REGISTRO_ACAO (
id_registro INTEGER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1),
id_acao INTEGER NOT NULL,
id_usuario INTEGER NOT NULL,
data_hora_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
status_validacao VARCHAR2(30) DEFAULT 'PENDENTE' NOT NULL,
pontos_obtidos INTEGER DEFAULT 0 NOT NULL,
latitude NUMBER(10,6),
longitude NUMBER(10,6),
CONSTRAINT PK_TB_REGISTRO_ACAO PRIMARY KEY (id_registro),
CONSTRAINT FK_REG_ACAO_SUST FOREIGN KEY (id_acao) REFERENCES TB_ACAO_SUSTENTAVEL (id_acao),
CONSTRAINT FK_REG_ACAO_USUARIO FOREIGN KEY (id_usuario) REFERENCES TB_USUARIO (id_usuario),
CONSTRAINT CK_REG_STATUS CHECK (status_validacao IN ('PENDENTE', 'VALIDADO', 'REJEITADO'))
);

CREATE TABLE TB_EVIDENCIA (
id_evidencia INTEGER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1),
id_registro INTEGER NOT NULL,
tipo_evidencia VARCHAR2(50) NOT NULL,
url_arquivo VARCHAR2(300) NOT NULL,
```

```
data_hora_envio TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,

descricao_evidencia VARCHAR2(300),

CONSTRAINT PK_TB_EVIDENCIA PRIMARY KEY (id_evidencia),

CONSTRAINT FK_EVID_REG_ACAO FOREIGN KEY (id_registro) REFERENCES TB_REGISTRO_ACAO (id_registro),

 CONSTRAINT CK_EVID_TIPO CHECK (tipo_evidencia IN ('FOTO', 'VIDEO', 'DOCUMENTO', 'QR_CODE'))

 );


CREATE TABLE TB_VALIDACAO (

id_validacao INTEGER GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1),

id_parceiro INTEGER NOT NULL,

id_registro INTEGER NOT NULL,

data_hora_validacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,

resultado_validacao VARCHAR2(30) NOT NULL,

observacao_validacao VARCHAR2(300),

CONSTRAINT PK_TB_VALIDACAO PRIMARY KEY (id_validacao),

CONSTRAINT UK_VALIDACAO_REGISTRO UNIQUE (id_registro),

CONSTRAINT FK_VALID_PARCEIRO FOREIGN KEY (id_parceiro) REFERENCES TB_PARCEIRO (id_parceiro),

CONSTRAINT FK_VALID_REG_ACAO FOREIGN KEY (id_registro) REFERENCES TB_REGISTRO_ACAO (id_registro),

 CONSTRAINT CK_VALID_RESULTADO CHECK (resultado_validacao IN ('APROVADO', 'REPROVADO'))

);
```

5. INTEGRAÇÃO JAVA X BANCO DE DADOS

A versão final foi ajustada para usar os nomes reais do modelo físico. Isso evita manter uma estrutura Java desconectada do banco desenvolvido na outra disciplina.

Java	Oracle
Usuario	TB_USUARIO
id	id_usuario
nome	nome_usuario
email	email_usuario
cpf	cpf_usuario
totalPontos	total_pontos_usuario
AcaoSustentavel	TB_ACAO_SUSTENTAVEL
pontuacaoBase	pontuacao_base
nivelDificuldade	nivel_dificuldade
RegistroAcao	TB_REGISTRO_ACAO
usuarioId / acaoId	id_usuario / id_acao
statusValidacao	status_validacao
pontosObtidos	pontos_obtidos

As demais tabelas do modelo físico permanecem no script oficial e poderão ser incorporadas ao Java nas próximas evoluções. Nesta Sprint, o CRUD foi aplicado às três classes centrais utilizadas pela aplicação.

6. CONEXÕES E FECHAMENTO DE RECURSOS

ConexaoOracleProjeto.java é a classe responsável pela conexão com o Oracle oficial da FIAP. A URL JDBC, o usuário Oracle da equipe e a senha estão configurados diretamente na classe, conforme solicitado para a Sprint. O driver ojdbc11 está declarado no pom.xml e os DAOs utilizam SQL compatível com os nomes do modelo físico.

A versão atual utiliza somente a conexão Oracle. O ambiente H2 utilizado durante a fase de desenvolvimento foi removido da versão destinada à entrega.

6.1.Fechamento explicito

Conforme a orientação dada em aula, todos os métodos DAO que abrem Connection fecham explicitamente ResultSet, PreparedStatement e Connection após o uso. O teste da conexão Oracle também fecha a Connection no bloco finally.

Os comentários de implementação foram concentrados somente nas duas classes de conexão. As demais explicações técnicas estão nesta documentação.

7. DAO E CRUD COMPLETO

DAO	Create	Read	Update	Delete
UsuarioDAO	criar()	buscarPorId()/listarTodos()	atualizar()	excluir()
AcaoSustentavelDAO	criar()	buscarPorId()/listarTodas()	atualizar()	excluir()
RegistroAcaoDAO	criar()	buscarPorId()/listarTodos()	atualizar()	excluir()

Os INSERTs não informam os IDs porque o modelo Oracle utiliza GENERATED ALWAYS AS IDENTITY. Após o CREATE, o DAO recupera a chave gerada e atualiza o objeto Java. Os UPDATEs e DELETEs utilizam a chave primária correspondente.

A ordem de exclusão respeita as chaves estrangeiras: primeiro o registro de ação, depois a ação sustentável e o usuário. Isso evita violação de integridade referencial durante os testes.

8. PROCEDIMENTOS PARA EXECUTAR

Ferramentas: IntelliJ IDEA ou IDE Java equivalente; Maven; JDK 17 ou superior.

8.1.Execução do projeto

- Abrir a pasta do projeto JavSprint3.
- Aguardar o Maven carregar as dependências.
- Abrir src/main/java/teste/Main.java.
- Executar o método main.
- Conferir no console o teste da conexão Oracle, as regras de negócio e os CRUDs de usuário, ação e registro.

8.2.Oracle oficial

- Executar database/script_sprint3_tetramerge.sql no schema Oracle da equipe.
- Abrir ConexaoOracleProjeto.java.
- Conferir os dados de URL, USUARIO e SENHA já configurados para o Oracle da equipe.
- Testar a conexão.
- Os três DAOs já utilizam ConexaoOracleProjeto para acessar o Oracle.
- Executar novamente o Main e validar o CRUD no schema oficial.

O projeto não contém README porque todas as instruções necessárias para a Sprint estão centralizadas nesta documentação.

9. VALIDAÇÃO FINAL E OBSERVAÇÕES

O código-fonte foi revisado e compilado com Java 17 sem erros de sintaxe. A estrutura foi mantida coerente com o script Oracle recebido, incluindo nomes de tabelas, colunas e uso de IDs gerados automaticamente. A execução integral foi realizada no ambiente da equipe com Maven, JDK 17 e Oracle.

O projeto inclui o script SQL oficial recebido da disciplina de Banco de Dados dentro da pasta database, permitindo ao avaliador conferir a correspondência entre o modelo físico e a implementação Java.

10. CONCLUSÃO

A conexão com o banco de dados Oracle está configurada no projeto. O Diagrama de Classes foi estruturado no Astah de acordo com as classes, atributos, métodos e relacionamentos implementados na aplicação. O projeto também foi validado no ambiente da equipe, considerando a utilização do JDK 17, Maven e Oracle, com verificação da compilação, da conexão com o banco de dados e das operações de CRUD implementadas.

11. REFERÊNCIAS

FORMAÇÃO SOCIAL E SUSTENTABILIDADE. Governança ambiental, social e corporativa (ESG). São Paulo: FIAP, 2022.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS (ONU). Transformando nosso mundo: a Agenda 2030 para o Desenvolvimento Sustentável. 2015.

FIAP. Java — Enunciado da Entrega Sprint 3. 2026.